Headers, cookies, sessions

Headers, cookies, and sessions **form the core mechanism for managing state and authentication in stateless HTTP web applications**.

1. HTTP Headers (The Control Plane)

HTTP headers are key-value pairs exchanged in request and response messages that pass metadata between the client and backend server.

Request Headers: Sent by the client to the server (e.g., `Cookie: session_id=abc123`, `Authorization: Bearer <token>`, `User-Agent`).

- **Request Headers:** Sent by the client to the server (e.g., `Cookie: session_id=abc123`, `Authorization: Bearer <token>`, `User-Agent`).

Response Headers: Sent by the server to the client (e.g., `Set-Cookie`, `WWW-Authenticate`, `Cache-Control`).

- **Response Headers:** Sent by the server to the client (e.g., `Set-Cookie`, `WWW-Authenticate`, `Cache-Control`).

Role in State: Because HTTP is stateless (each request is independent), headers carry the tokens or identifiers the server needs to recognize the user.

- **Role in State:** Because HTTP is stateless (each request is independent), headers carry the tokens or identifiers the server needs to recognize the user.

---

2. Cookies (The Browser Storage Mechanism)

A cookie is a small piece of data (up to 4KB) that the server tells the browser to store using the `Set-Cookie` response header.

Automatic Transmission: Once set, the browser automatically attaches the cookie to every subsequent request directed to that domain via the `Cookie` request header.

- **Automatic Transmission:** Once set, the browser automatically attaches the cookie to every subsequent request directed to that domain via the `Cookie` request header.

Key `Set-Cookie` Attributes:

◦ `HttpOnly`: Prevents client-side JavaScript (like `document.cookie`) from accessing the cookie, mitigating Cross-Site Scripting (XSS) theft.

◦ `Secure`: Ensures the cookie is only transmitted over encrypted HTTPS connections.

◦ `SameSite=Strict | Lax | None`: Controls whether the cookie is sent on cross-site requests to help prevent Cross-Site Request Forgery (CSRF).

◦ `Max-Age` / `Expires`: Defines how long the cookie persists in the browser before expiring.

- **Key `Set-Cookie` Attributes:**
  - ◦ `HttpOnly`: Prevents client-side JavaScript (like `document.cookie`) from accessing the cookie, mitigating Cross-Site Scripting (XSS) theft.
  - ◦ `Secure`: Ensures the cookie is only transmitted over encrypted HTTPS connections.
  - ◦ `SameSite=Strict | Lax | None`: Controls whether the cookie is sent on cross-site requests to help prevent Cross-Site Request Forgery (CSRF).
  - ◦ `Max-Age` / `Expires`: Defines how long the cookie persists in the browser before expiring.

---

3. Sessions (The Server-Side State)

HTTP is stateless, so a **session** allows the backend to remember who a user is across multiple requests.

How it works:

1. The user logs in with credentials.

2. The backend creates a session record (storing user ID, role, and metadata) in a server-side **session store** (such as Redis, a database, or in-memory).

3. The backend issues a unique, random **session ID** back to the client via a `Set-Cookie` header (e.g., `session_id=xyz789`).

4. On every future request, the browser sends `session_id=xyz789` in the `Cookie` header; the backend looks up `xyz789` in its session store to identify the user.

- **How it works:**
  1. 1. The user logs in with credentials.

2. 2. The backend creates a session record (storing user ID, role, and metadata) in a server-side **session store** (such as Redis, a database, or in-memory).
3. 3. The backend issues a unique, random **session ID** back to the client via a `Set-Cookie` header (e.g., `session_id=xyz789`).
4. 4. On every future request, the browser sends `session_id=xyz789` in the `Cookie` header; the backend looks up `xyz789` in its session store to identify the user.

1. **Stateful vs. Stateless Alternatives:**
   ◦ **Server-side Sessions:** State lives on the backend database/memory; revoking a session is instant (just delete the record).
   ◦ **Token-based / JWT:** State is encoded directly inside a signed token given to the client; the server doesn't need to store session data in a database, making it more scalable for microservices and mobile apps, though harder to revoke before expiration

- **Stateful vs. Stateless Alternatives:**
  - ◦ **Server-side Sessions:** State lives on the backend database/memory; revoking a session is instant (just delete the record).
  - ◦ **Token-based / JWT:** State is encoded directly inside a signed token given to the client; the server doesn't need to store session data in a database, making it more scalable for microservices and mobile apps, though harder to revoke before expiration